

[YOUR COLLEGE NAME]
DEPARTMENT OF [YOUR DEPARTMENT]

Project Modules & Implementation Plan

On

AI-Powered Smart Farming & Gardening Assistant

Submitted in partial fulfillment of the requirements for
9th Semester Mini Project

Submitted by:

[Student Name 1] – [Register Number]

[Student Name 2] – [Register Number]

[Student Name 3] – [Register Number]

Under the Guidance of:

[Guide Name]

[Designation]

[Month, Year]

Contents

1	Introduction	2
2	Project Modules	2
2.1	Module 1: Smart Farming	2
2.2	Module 2: Smart Gardening	2
2.3	Module 3: Shared Core Infrastructure	2
2.4	Module 4: AI Assistant	3
2.5	Module Summary Table	3
3	Team Role Distribution (3 Members)	3
4	Phase-wise Implementation Plan	4
4.1	Overview	4
4.2	Timeline (Gantt Chart)	5
5	Detailed Phase Descriptions	5
6	Risk Assessment & Mitigation	8
7	Conclusion	8

1 Introduction

This document defines the functional modules of the *AI-Powered Smart Farming & Gardening Assistant* and lays out a phase-wise implementation plan for a 3-member team over a full semester (16 weeks). The plan is designed so that Machine Learning, Backend/IoT, and Frontend work streams progress in parallel where possible, converging at defined integration and review checkpoints.

2 Project Modules

2.1 Module 1: Smart Farming

- Crop Recommendation (ML classification on soil/weather parameters)
- Yield Prediction (ML regression)
- Weather Dashboard (API-based forecast, rainfall trend)
- Leaf Disease Detection (CNN, shared with Module 2)
- Smart Irrigation Alerts (soil moisture + weather-based)
- Farm Dashboard (crop tracking, sensor readings, alerts)

2.2 Module 2: Smart Gardening

- Plant Identification from Image (CNN, transfer learning)
- Plant Care Information (watering, sunlight, soil requirements)
- Plant Disease Detection (shared model from Module 1)
- Smart Pot Monitoring (same IoT hardware, reframed use case)
- Gardening Dashboard (plant collection, care history)

2.3 Module 3: Shared Core Infrastructure

- Authentication & Role Management (Farmer / Gardener / Admin)
- IoT Data Ingestion Layer (MQTT/HTTP → backend → database)
- ML Model Serving Layer (FastAPI inference endpoints)
- Weather & Location API Integration
- Common Dashboard Framework (charts, alerts, analytics)

2.4 Module 4: AI Assistant

- Explains model predictions in simple language (e.g., "why this crop?")
- Answers general farming/gardening queries
- Surfaces personalized recommendations based on user's farm/plant data

2.5 Module Summary Table

Module	Core Purpose	Primary Tech
Smart Farming	Crop/yield decisions, disease & irrigation alerts for farmers	Scikit-learn, TensorFlow, FastAPI
Smart Gardening	Plant ID, care guidance, disease detection for home gardeners	TensorFlow (transfer learning), React Native
Shared Infrastructure	Auth, IoT ingestion, model serving, common dashboard	FastAPI, PostgreSQL, MQTT
AI Assistant	Explainable, conversational layer over predictions	SHAP/LIME, prompt templates

3 Team Role Distribution (3 Members)

Member	Primary Responsibility	Phases Involved
Member A	AI/ML & Deep Learning Engineer – dataset prep, model training, evaluation, explainability	Phases 3, 4, 5, 6, 10
Member B	Backend & IoT Engineer – API development, database, hardware/firmware, deployment	Phases 2, 7, 8, 11, 13
Member C	Frontend & UI/UX Lead – mobile/web app, dashboards, documentation, presentation	Phases 2, 9, 12, 14

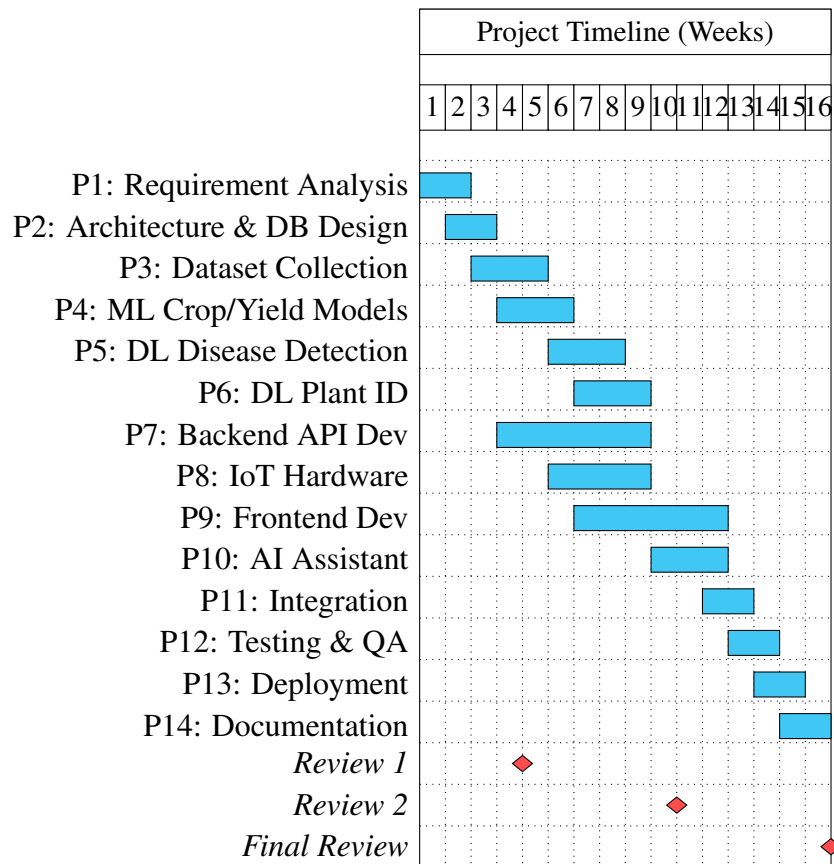
Note: Phases 1, 11, 12, and 14 (planning, integration, testing, documentation) are collaborative and involve all members.

4 Phase-wise Implementation Plan

4.1 Overview

#	Phase	Weeks	Key Deliverable
1	Requirement Analysis & Planning	1–2	SRS document, finalized feature list
2	System Architecture & Database Design	2–3	Architecture diagram, ER diagram, API spec
3	Dataset Collection & Pre-processing	3–5	Cleaned, augmented datasets + EDA report
4	ML Models: Crop Recommendation & Yield Prediction	4–6	Trained models with evaluation report
5	DL Model: Leaf/Plant Disease Detection (CNN)	6–8	Trained CNN, confusion matrix, exported model
6	DL Model: Plant Species Identification (CNN)	7–9	Trained plant ID model
7	Backend API Development	4–9	Working REST API with Swagger docs
8	IoT Hardware Development & Firmware	6–9	ESP32 prototype streaming live sensor data
9	Frontend Development (Mobile + Web)	7–12	Functional cross-platform app
10	AI Assistant / Explainability Integration	10–12	Assistant chat interface in dashboard
11	System Integration	12–13	Fully integrated working prototype
12	Testing & Quality Assurance	13–14	Test report, resolved bug log
13	Deployment	14–15	Live hosted system + installable app
14	Documentation & Presentation Preparation	15–16	Final report, slides, demo script

4.2 Timeline (Gantt Chart)



5 Detailed Phase Descriptions

Phase 1: Requirement Analysis & Planning (Week 1–2)

Objective: Finalize project scope, features (MVP vs. future scope), and technology stack.

Tasks: Literature study; finalize functional/non-functional requirements; assign team roles; set up GitHub repository and project board.

Tools: GitHub, Notion/Google Docs.

Deliverable: SRS document, finalized feature list, GitHub repo initialized.

Phase 2: System Architecture & Database Design (Week 2–3)

Objective: Define the overall system architecture and data model.

Tasks: Design client-server-ML-IoT architecture diagram; design ER diagram (Users, Farms, Plants, SensorReadings, Predictions, Alerts); define REST API contract.

Tools: draw.io, PostgreSQL, Postman.

Deliverable: Architecture diagram, ER diagram, API specification document.

Phase 3: Dataset Collection & Preprocessing (Week 3–5)

Objective: Prepare all datasets required for model training.

Tasks: Collect crop recommendation dataset, PlantVillage disease dataset, curated regional plant-species images, historical weather data; clean and normalize; perform train/val/test split; apply image augmentation.

Tools: Python, Pandas, OpenCV, Kaggle API.

Deliverable: Cleaned, ready-to-train datasets and an exploratory data analysis (EDA) report.

Phase 4: ML Models – Crop Recommendation & Yield Prediction (Week 4–6)

Objective: Build the two core classical-ML models.

Tasks: Train/evaluate Random Forest and Gradient Boosting classifiers for crop recommendation; train a regression model for yield prediction; add SHAP/LIME-based explainability.

Tools: Scikit-learn, XGBoost, SHAP, Google Colab.

Deliverable: Trained model files (.pkl), accuracy/precision/recall report, explainability outputs.

Phase 5: DL Model – Leaf/Plant Disease Detection (Week 6–8)

Objective: Build the CNN shared between Smart Farming and Smart Gardening.

Tasks: Fine-tune a transfer-learning model (e.g., MobileNetV2/ResNet) on PlantVillage plus supplementary field images; evaluate with confusion matrix; export to TensorFlow Lite/ONNX for lightweight serving.

Tools: TensorFlow/Keras or PyTorch, Google Colab GPU.

Deliverable: Trained disease-detection model with evaluation metrics.

Phase 6: DL Model – Plant Species Identification (Week 7–9)

Objective: Build the plant-identification CNN for Smart Gardening.

Tasks: Curate a regional plant-species subset; apply transfer learning; evaluate top-1/top-5 accuracy.

Tools: TensorFlow/Keras, Google Colab GPU.

Deliverable: Trained plant identification model.

Phase 7: Backend API Development (Week 4–9, parallel)

Objective: Build the central FastAPI backend.

Tasks: Set up project structure; implement JWT authentication and role management; build CRUD APIs for farms/plants/sensor data; wrap trained ML models as inference endpoints; integrate weather/maps APIs.

Tools: FastAPI, PostgreSQL, Uvicorn, JWT.

Deliverable: Working backend with Swagger-documented endpoints.

Phase 8: IoT Hardware Development & Firmware (Week 6–9, parallel)

Objective: Build the physical sensor node.

Tasks: Assemble ESP32 with soil moisture, DHT22, LDR, and rain sensors; write firmware to publish readings via MQTT/HTTP; calibrate thresholds; test connectivity to backend.

Tools: Arduino IDE/PlatformIO, ESP32, Mosquitto MQTT broker.

Deliverable: Working IoT prototype streaming live data to the backend.

Phase 9: Frontend Development – Mobile + Web (Week 7–12)

Objective: Build the user-facing application.

Tasks: Set up Expo + React Native + React Native Web; build Farmer and Gardener dashboards; implement camera upload for disease/plant detection; integrate charts for sensor/prediction data; connect to backend via React Query; implement push notifications.

Tools: Expo, NativeWind, React Navigation, TanStack Query, Victory Native/Recharts.

Deliverable: Functional cross-platform application (mobile + web build).

Phase 10: AI Assistant / Explainability Integration (Week 10–12)

Objective: Add the conversational, explainable layer.

Tasks: Convert model outputs and SHAP/LIME explanations into plain-language responses; build a simple chat interface; optionally connect a free-tier LLM API for general Q&A.

Tools: Python, prompt templates, FastAPI endpoint.

Deliverable: Working AI Assistant embedded in the dashboard.

Phase 11: System Integration (Week 12–13)

Objective: Connect all modules end-to-end.

Tasks: Wire frontend ↔ backend ↔ ML models ↔ IoT pipeline; resolve data-flow mismatches; verify role-based access across the app.

Deliverable: Fully integrated working prototype.

Phase 12: Testing & Quality Assurance (Week 13–14)

Objective: Validate correctness and stability.

Tasks: Unit tests for backend and models; integration tests for end-to-end flows; informal user testing with a small farmer/gardener group if possible.

Tools: Pytest, Postman.

Deliverable: Test report and resolved bug log.

Phase 13: Deployment (Week 14–15)

Objective: Make the system publicly accessible for demo/evaluation.

Tasks: Deploy backend to Render/Railway; host database on Supabase; deploy web build to Vercel; build mobile app via Expo EAS.

Deliverable: Live hosted system and an installable/demo-ready mobile app.

Phase 14: Documentation & Presentation Preparation (Week 15–16)

Objective: Prepare all submission and evaluation material.

Tasks: Compile final project report; prepare presentation slides and exhibition poster; rehearse demo and anticipated viva questions.

Deliverable: Final report, presentation deck, demo script.

6 Risk Assessment & Mitigation

Risk	Likelihood/Impact	Mitigation
Low model accuracy on field images (lab-to-field gap)	Medium / High	Augment training data with self-captured field images early (Phase 3)
IoT hardware delays (shipping, faulty components)	Medium / Medium	Order components by Week 2; keep a software-simulated sensor fallback
Scope creep (too many features)	High / High	Strictly follow MVP list; treat stretch/future features as optional
Integration issues late in timeline	Medium / High	Start backend-frontend integration incrementally from Phase 7, not only in Phase 11
Free-tier hosting limits during demo	Low / Medium	Test deployment early (Phase 13 dry-run in Week 12)

7 Conclusion

This phase-wise plan distributes AI/ML, backend/IoT, and frontend work across a 16-week semester with three built-in review checkpoints, allowing parallel progress while ensuring in-

tegration and testing are never compressed into the final days. Following this plan keeps the MVP scope achievable while leaving clear room for stretch features if time permits.